

MECHATRONICS AND IOT

ASSIGNMENT REPORT - 15

TITLE:

Design and develop an Automated Color-Based Product Sorting System for Manufacturing Industries. The system automatically detects and sorts products based on color for industrial automation applications

DONE BY ,

HARI KRISHNA JAGADEESH-24MER011

HARISH DHARMALINGAM – 24MER014

RANJITH KUMAR P – 24MER048

RISHI KESAV A – 24MER049

VENKATRAMANAN B – 24MER061

Project Overview:

The Automated Color-Based Product Sorting System is an industrial automation system designed to automatically detect and sort products based on their color. The system uses a color sensor to identify the color of products moving along a conveyor or placed in the sorting area.

A microcontroller such as ESP32/Arduino processes the color sensor readings and determines the corresponding product category. Based on the detected color, a servo motor or actuator directs the product into the appropriate sorting bin.

The system can use IR sensors to detect the presence of products and initiate the sorting process automatically. LEDs or an OLED display can also be used to indicate the detected color and sorting status.

The proposed system provides a low-cost, fast and automated solution for color-based product classification, reducing manual sorting effort and improving sorting accuracy, productivity and efficiency in manufacturing industries.

Problem Statement:

In manufacturing industries, products often need to be sorted and classified based on their color before packaging or further processing. Traditional manual sorting methods are time-consuming, labor-intensive and prone to human errors, especially when large quantities of products need to be processed.

Inconsistent sorting can lead to product misclassification, reduced productivity and increased operational costs. Therefore, there is a need for an automated color-based sorting system that can accurately detect product colors and automatically direct them to the appropriate sorting category.

The proposed system aims to reduce manual intervention, improve sorting accuracy and speed, and enhance overall production efficiency in industrial automation applications.

Proposed Solution:

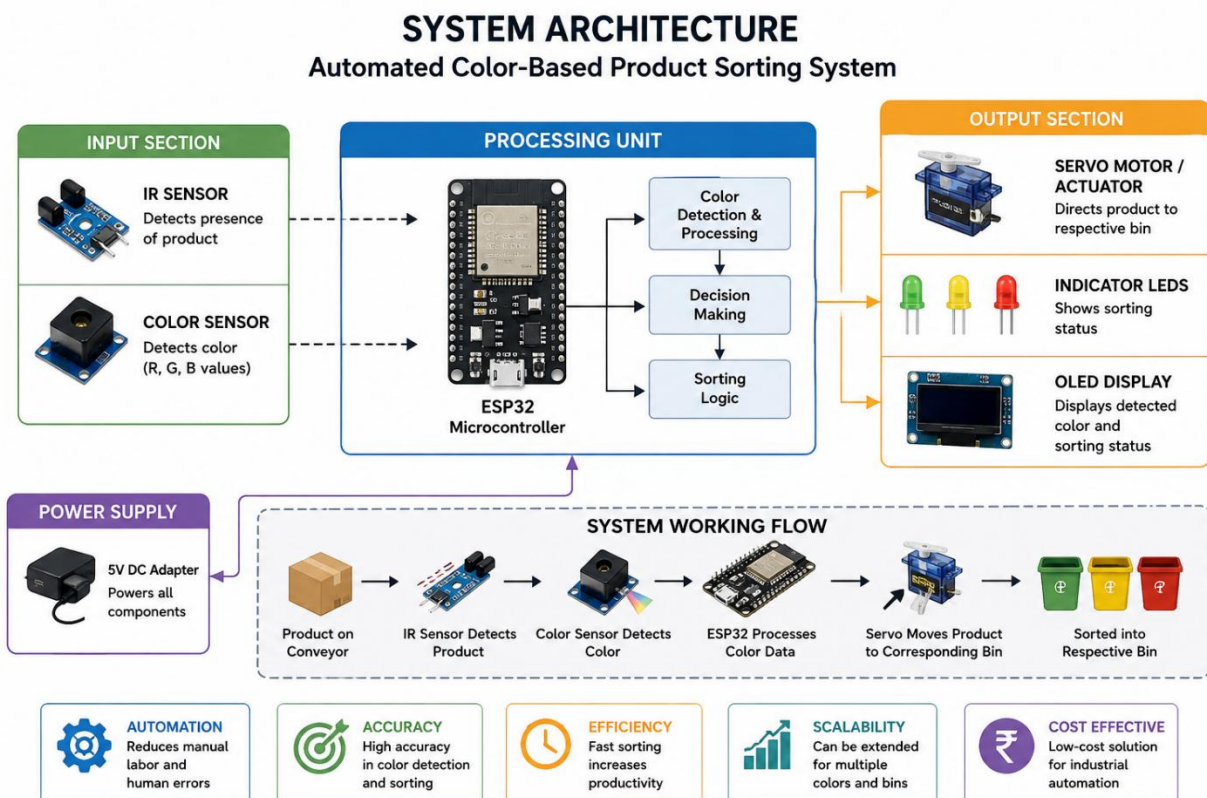
The proposed system uses an ESP32/Arduino microcontroller, color sensor, IR sensor and servo motor to automatically detect and sort products based on their color. The IR sensor detects the presence of a product, while the color sensor identifies its color.

The microcontroller processes the sensor readings and compares the detected color with predefined color categories. Based on the identified color, a servo motor or actuator directs the product into the corresponding sorting bin.

An OLED display can be used to show the detected color and sorting status, while LEDs can provide visual indications of the sorting operation. The system can be mounted on a small conveyor mechanism to demonstrate continuous automated sorting.

The proposed solution provides a low-cost, fast and automated sorting system that reduces manual intervention, improves sorting accuracy and increases productivity in manufacturing industries.

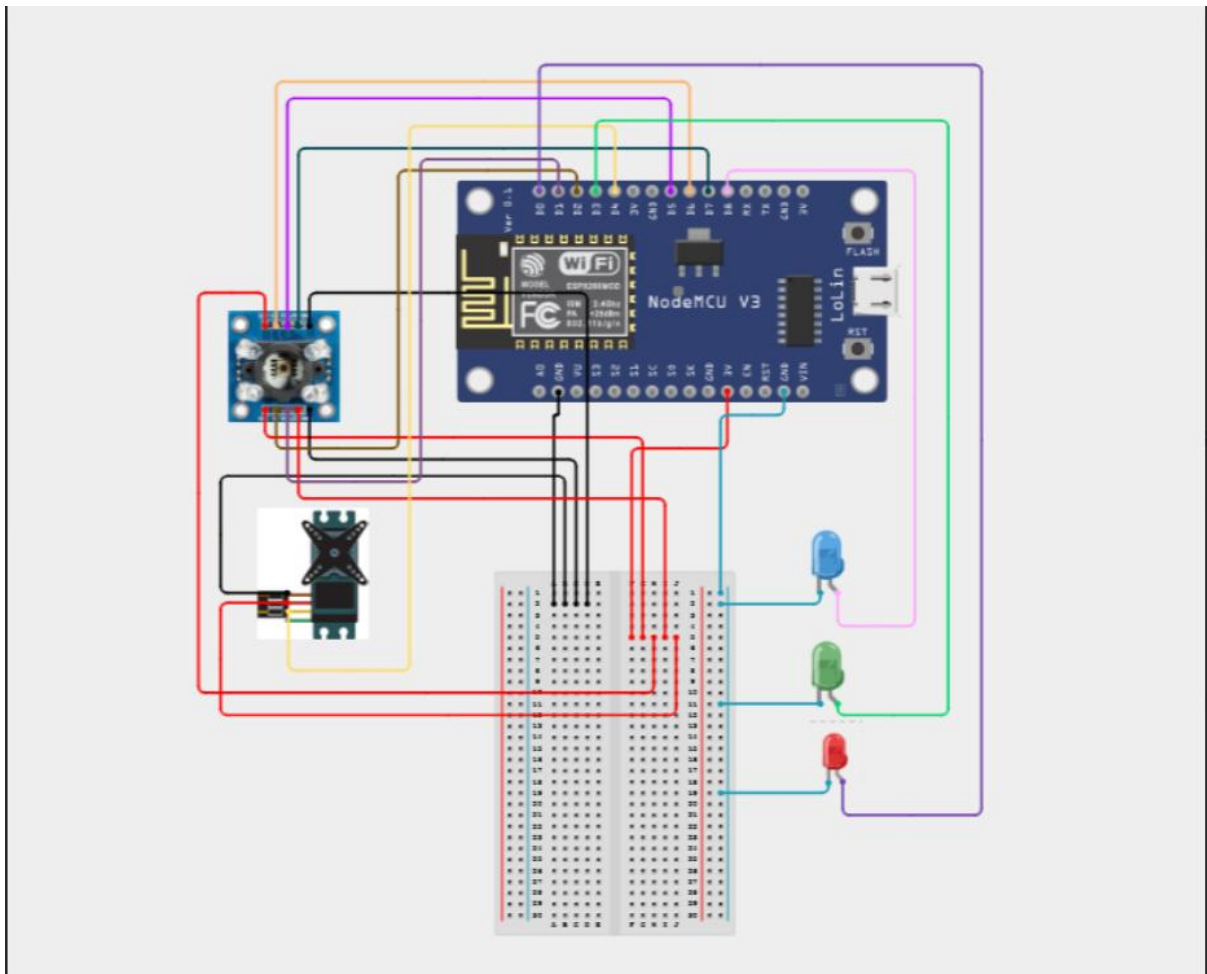
System Architecture:



Circuit Table :

Component	Pin/Terminal	ESP8266 Connection	Purpose
Color Sensor (TCS3200)	VCC	3.3V	Power supply
ESP8266	VIN/USB	5V USB	Hazardous indication
LED	Anode (+)	GPIO	Indicates one color/category
Servo Motor	VCC	External 5V	Moves product to correct bin

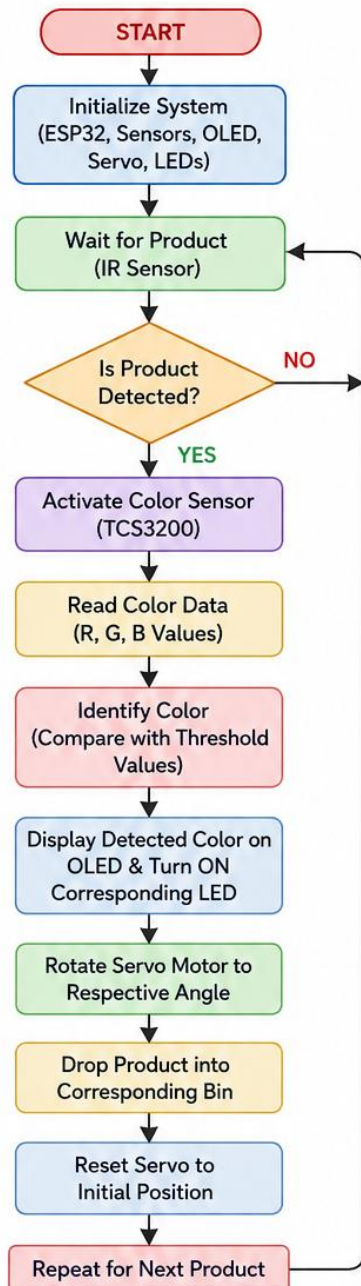
Circuit Design:



Algorithm:

ALGORITHM

AUTOMATED COLOR-BASED PRODUCT SORTING SYSTEM



STEP-BY-STEP ALGORITHM

- 1 Start the system.
- 2 Initialize ESP32, IR Sensor, Color Sensor, Servo Motor, OLED Display and LEDs.
- 3 Continuously wait for a product. (IR Sensor monitors the product presence)
- 4 If a product is not detected, continue waiting. If detected, go to next step.
- 5 Activate the Color Sensor (TCS3200) to sense the color.
- 6 Read the Red, Green and Blue (RGB) values from the sensor.
- 7 Compare the RGB values with predefined threshold values.
- 8 Identify the color of the product (e.g., Red / Green / Blue / Yellow).
- 9 Display the detected color on OLED and turn ON the corresponding LED.
- 10 Rotate the Servo Motor to the angle assigned to that color.
- 11 The product falls into the corresponding bin.
- 12 After a short delay, reset the servo to initial position.
- 13 Repeat the process for the next product.



NOTE:

- Threshold values for colors are calibrated before operation.
- Servo angles can be adjusted as per mechanical setup.
- Add delay (e.g., 500 ms) between sorting consecutive products for accuracy.

COLOR – BIN MAPPING (Example)

RED	→	Bin 1	→	Servo Angle : 0°
GREEN	→	Bin 2	→	Servo Angle : 60°
BLUE	→	Bin 3	→	Servo Angle : 120°
YELLOW	→	Bin 4	→	Servo Angle : 180°

Coding:

AUTOMATED COLOR-BASED PRODUCT SORTING SYSTEM

ESP8266 + TCS3200 + 3 LEDs + Servo + Adafruit IO

COLOR DETECTION:

Red > 2000 Hz

Green > 2000 Hz

Blue > 2000 Hz

SERVO:

No color = 0 degrees

Color detected = 180 degrees

ADAFRUIT IO:

Updates every 30 seconds

FEEDS:

red-value

green-value

blue-value

detected-color

red-count

green-count

blue-count

servo-position

sorting-count

system-status

*****/

```
#include <ESP8266WiFi.h>
```

```
#include "AdafruitIO_WiFi.h"
```

```
#include <Servo.h>
```

```
// =====
```

```

// WIFI SETTINGS

// =====

#define WIFI_SSID    "Harish"

#define WIFI_PASS    "Harish@1234"

// =====

// ADAFRUIT IO SETTINGS

// =====

#define IO_USERNAME  "HARISH_2007"

#define IO_KEY       "aio_ZJNX14DMuLnYEh9CihQDt0MlBdmE"

AdafruitIO_WiFi io(IO_USERNAME, IO_KEY, WIFI_SSID, WIFI_PASS);

// =====

// TCS3200 PINS

// =====

// GPIO numbers are used instead of D1/D2 etc.

#define S0_PIN    5    // D1

#define S1_PIN    4    // D2

#define S2_PIN    14   // D5

#define S3_PIN    12   // D6

#define OUT_PIN   13   // D7

// =====

// LED PINS

// =====

#define RED_LED    16   // D0

#define GREEN_LED  0    // D3

#define BLUE_LED   15   // D8

// =====

// SERVO

// =====

#define SERVO_PIN  2    // D4

Servo sortingServo;

```

```
// =====  
// COLOR DETECTION THRESHOLD  
// =====  
#define COLOR_THRESHOLD 2000  
// =====  
// ADAFRUIT IO FEEDS  
// =====  
AdafruitIO_Feed *redValueFeed =  
    io.feed("red-value");  
AdafruitIO_Feed *greenValueFeed =  
    io.feed("green-value");  
AdafruitIO_Feed *blueValueFeed =  
    io.feed("blue-value");  
AdafruitIO_Feed *detectedColorFeed =  
    io.feed("detected-color");  
AdafruitIO_Feed *redCountFeed =  
    io.feed("red-count");  
AdafruitIO_Feed *greenCountFeed =  
    io.feed("green-count");  
AdafruitIO_Feed *blueCountFeed =  
    io.feed("blue-count");  
AdafruitIO_Feed *servoPositionFeed =  
    io.feed("servo-position");  
AdafruitIO_Feed *sortingCountFeed =  
    io.feed("sorting-count");  
AdafruitIO_Feed *systemStatusFeed =  
    io.feed("system-status");  
// =====  
// VARIABLES  
// =====
```



```
unsigned long redHz = 0;
unsigned long greenHz = 0;
unsigned long blueHz = 0;

// =====

// COUNTERS

// =====

int redCount = 0;
int greenCount = 0;
int blueCount = 0;
int sortingCount = 0;

// =====

// CURRENT STATUS

// =====

String detectedColor = "NONE";
int currentServoPosition = 0;

// =====

// TIMERS

// =====

// Adafruit update every 30 seconds
unsigned long lastAdafruitUpdate = 0;
const unsigned long ADAFRUIT_INTERVAL = 30000;

// =====

// PRODUCT DETECTION LOCK

// =====

// Prevents the same object from being counted
// continuously while it remains in front of sensor.

bool productDetected = false;

// =====

// READ TCS3200 FREQUENCY

// =====
```

```
unsigned long readFrequency(  
    bool s2State,  
    bool s3State  
)  
{  
    digitalWrite(S2_PIN, s2State);  
    digitalWrite(S3_PIN, s3State);  
    delay(30);  
    unsigned long highTime =  
        pulseIn(  
            OUT_PIN,  
            HIGH,  
            100000  
        );  
    unsigned long lowTime =  
        pulseIn(  
            OUT_PIN,  
            LOW,  
            100000  
        );  
    unsigned long period =  
        highTime + lowTime;  
    if (period == 0)  
    {  
        return 0;  
    }  
    unsigned long frequency =  
        1000000UL / period;  
    return frequency;  
}
```

```
// =====  
  
// READ RED  
  
// =====  
  
unsigned long readRed()  
{  
    // S2 LOW  
    // S3 LOW  
    return readFrequency(  
        LOW,  
        LOW  
    );  
}  
  
// =====  
  
// READ GREEN  
  
// =====  
  
unsigned long readGreen()  
{  
    // S2 HIGH  
    // S3 HIGH  
    return readFrequency(  
        HIGH,  
        HIGH  
    );  
}  
  
// =====  
  
// READ BLUE  
  
// =====  
  
unsigned long readBlue()  
{  
    // S2 LOW
```

```
// S3 HIGH
return readFrequency(
    LOW,
    HIGH
);
}
// =====

// TURN OFF ALL LEDS
// =====

void allLEDsOff()
{
    digitalWrite(
        RED_LED,
        LOW
    );
    digitalWrite(
        GREEN_LED,
        LOW
    );
    digitalWrite(
        BLUE_LED,
        LOW
    );
}
// =====

// RED LED
// =====

void redLED()
{
    allLEDsOff();
}
```

```
digitalWrite(
  RED_LED,
  HIGH
);
}
// =====
// GREEN LED
// =====

void greenLED()
{
  allLEDsOff();
  digitalWrite(
    GREEN_LED,
    HIGH
  );
}
// =====
// BLUE LED
// =====

void blueLED()
{
  allLEDsOff();
  digitalWrite(
    BLUE_LED,
    HIGH
  );
}
// =====
// SERVO NORMAL POSITION
// =====
```

```

void servoNormal()
{
    currentServoPosition = 0;
    sortingServo.write(0);
}
// =====
// SERVO SORTING POSITION
// =====

void servoSort()
{
    currentServoPosition = 180;
    sortingServo.write(180);
}
// =====
// DETECT COLOR
// =====

String detectColor()
{
    // Read all three channels
    redHz =
        readRed();
    greenHz =
        readGreen();
    blueHz =
        readBlue();
    // Print readings
    Serial.println();
    Serial.println(
        "=====")
);

```

```
Serial.print(
  "RED Hz  : "
);
Serial.println(redHz);
Serial.print(
  "GREEN Hz : "
);
Serial.println(greenHz);
Serial.print(
  "BLUE Hz : "
);
Serial.println(blueHz);
Serial.println(
  "Threshold: 2000 Hz"
);
// -----
// Find highest frequency
// -----
unsigned long highestHz =
  max(
    redHz,
    max(
      greenHz,
      blueHz
    )
  );
// -----
// Nothing detected
// -----
if (highestHz <= COLOR_THRESHOLD)
```

```
{
    return "NONE";
}

// -----

// RED

// -----

if (
    redHz > COLOR_THRESHOLD &&
    redHz >= greenHz &&
    redHz >= blueHz
)
{
    return "RED";
}

// -----

// GREEN

// -----

if (
    greenHz > COLOR_THRESHOLD &&
    greenHz >= redHz &&
    greenHz >= blueHz
)
{
    return "GREEN";
}

// -----

// BLUE

// -----

if (
    blueHz > COLOR_THRESHOLD &&
```



```

    blueHz >= redHz &&
    blueHz >= greenHz
)
{
    return "BLUE";
}
return "NONE";
}
// =====
// PROCESS NEW PRODUCT
// =====

void processNewProduct(
    String color
)
{
    Serial.println();
    Serial.println(
        "***** PRODUCT DETECTED *****"
    );
    Serial.print(
        "COLOR: "
    );
    Serial.println(color);
    // =====
    // RED PRODUCT
    // =====

    if (color == "RED")
    {
        redLED();
        servoSort();
    }
}

```

```
redCount++;
sortingCount++;
detectedColor = "RED";
Serial.println(
    "RED LED ON"
);
Serial.println(
    "SERVO = 180 degrees"
);
Serial.print(
    "RED COUNT = "
);
Serial.println(redCount);
}
// =====
// GREEN PRODUCT
// =====
else if (color == "GREEN")
{
    greenLED();
    servoSort();
    greenCount++;
    sortingCount++;
    detectedColor = "GREEN";
    Serial.println(
        "GREEN LED ON"
    );
    Serial.println(
        "SERVO = 180 degrees"
    );
```

```
Serial.print(  
    "GREEN COUNT = "  
);  
Serial.println(greenCount);  
}  
// =====  
// BLUE PRODUCT  
// =====  
else if (color == "BLUE")  
{  
    blueLED();  
    servoSort();  
    blueCount++;  
    sortingCount++;  
    detectedColor = "BLUE";  
    Serial.println(  
        "BLUE LED ON"  
    );  
    Serial.println(  
        "SERVO = 180 degrees"  
    );  
    Serial.print(  
        "BLUE COUNT = "  
    );  
    Serial.println(blueCount);  
}  
Serial.print(  
    "TOTAL SORTING COUNT = "  
);  
Serial.println(sortingCount);
```

```

Serial.println(
    "*****"
);
}
// =====
// RESET WHEN PRODUCT IS REMOVED
// =====

void resetProductDetection()
{
    productDetected = false;
    detectedColor = "NONE";
    allLEDsOff();
    servoNormal();
    Serial.println();
    Serial.println(
        "Product removed."
    );
    Serial.println(
        "Servo = 0 degrees"
    );
    Serial.println(
        "System ready for next product."
    );
}
// =====
// SEND DATA TO ADAFRUIT IO
// =====

void updateAdafruit()
{
    Serial.println();

```

```
Serial.println(
    "=====")
);
Serial.println(
    "UPDATING ADAFRUIT IO"
);
Serial.println(
    "=====")
);
// -----
// COLOR SENSOR VALUES
// -----
redValueFeed->save(
    (int)redHz
);
greenValueFeed->save(
    (int)greenHz
);
blueValueFeed->save(
    (int)blueHz
);
// -----
// DETECTED COLOR
// -----
detectedColorFeed->save(
    detectedColor.c_str()
);
// -----
// COUNTS
// -----
```

```
redCountFeed->save(  
    redCount  
);  
greenCountFeed->save(  
    greenCount  
);  
blueCountFeed->save(  
    blueCount  
);  
// -----  
// SERVO POSITION  
// -----  
servoPositionFeed->save(  
    currentServoPosition  
);  
// -----  
// TOTAL SORTING COUNT  
// -----  
sortingCountFeed->save(  
    sortingCount  
);  
// -----  
// SYSTEM STATUS  
// -----  
systemStatusFeed->save(  
    "ONLINE"  
);  
Serial.println(  
    "Adafruit IO update completed."  
);
```

```

Serial.println(
    "Next update after 30 seconds."
);
Serial.println(
    "=====
");
);
}
// =====
// SETUP
// =====

void setup()
{
    Serial.begin(115200);
    delay(1000);
    // =====
    // TCS3200
    // =====

    pinMode(
        S0_PIN,
        OUTPUT
    );
    pinMode(
        S1_PIN,
        OUTPUT
    );
    pinMode(
        S2_PIN,
        OUTPUT
    );
    pinMode(

```

```
S3_PIN,
    OUTPUT
);
pinMode(
    OUT_PIN,
    INPUT
);
// =====
// 20% FREQUENCY SCALING
// =====
digitalWrite(
    S0_PIN,
    HIGH
);
digitalWrite(
    S1_PIN,
    LOW
);
// =====
// LEDs
// =====
pinMode(
    RED_LED,
    OUTPUT
);
pinMode(
    GREEN_LED,
    OUTPUT
);
pinMode(
```



```
BLUE_LED,
OUTPUT
);
allLEDsOff();

// =====

// SERVO

// =====

sortingServo.attach(
SERVO_PIN
);
servoNormal();

// =====

// START MESSAGE

// =====

Serial.println();
Serial.println(
"=====")
);
Serial.println(
" AUTOMATED COLOR SORTING SYSTEM"
);
Serial.println(
"=====")
);
Serial.println();
Serial.println(
"Color threshold = 2000 Hz"
);
Serial.println(
"Servo normal   = 0 degrees"
```

```
);  
Serial.println(  
  "Servo sorting  = 180 degrees"  
);  
// =====  
// CONNECT ADAFRUIT IO  
// =====  
Serial.println();  
Serial.println(  
  "Connecting to Adafruit IO..."  
);  
io.connect();  
while (  
  io.status() < AIO_CONNECTED  
)  
{  
  Serial.print(  
    "."  
  );  
  delay(500);  
}  
Serial.println();  
Serial.println(  
  "Adafruit IO Connected!"  
);  
// =====  
// INITIAL ADAFRUIT VALUES  
// =====  
redValueFeed->save(0);
```

```
greenValueFeed->save(0);
blueValueFeed->save(0);
detectedColorFeed->save(
    "NONE"
);
redCountFeed->save(
    redCount
);
greenCountFeed->save(
    greenCount
);
blueCountFeed->save(
    blueCount
);
servoPositionFeed->save(0);
sortingCountFeed->save(
    sortingCount
);
systemStatusFeed->save(
    "ONLINE"
);
lastAdafruitUpdate =
    millis();
Serial.println();
Serial.println(
    "SYSTEM READY"
);
Serial.println(
    "Place a product in front of sensor."
);
```

```
Serial.println();

}

// =====

// MAIN LOOP

// =====

void loop()

{

    // Maintain Adafruit IO connection

    io.run();

    // =====

    // READ COLORS

    // =====

    String color =

        detectColor();

    // =====

    // PRODUCT DETECTED

    // =====

    if (

        color != "NONE" &&

        productDetected == false

    )

    {

        productDetected = true;

        processNewProduct(

            color

        );

    }

    // =====

    // PRODUCT REMOVED

    // =====
```

```

if (
    color == "NONE" &&
    productDetected == true
)
{
    resetProductDetection();
}

// =====
// ADAFRUIT IO EVERY 30 SECONDS
// =====

if (
    millis() -
    lastAdafruitUpdate >=
    ADAFRUIT_INTERVAL
)
{
    updateAdafruit();
    lastAdafruitUpdate =
        millis();
}

// Small delay
delay(300);
}

```

System Working:

The Automated Color-Based Product Sorting System works by continuously detecting and sorting products according to their color. When the system is powered ON, the ESP32, IR sensor, color sensor, OLED display and servo motor are initialized.

The IR sensor detects the presence of a product, after which the TCS3200 color sensor detects the product's color by measuring its RGB values. The ESP32 processes these

values and compares them with predefined color thresholds to identify the product color.

The detected color and sorting status are displayed on the OLED display, while the corresponding LED provides a visual indication. Based on the detected color, the ESP32 commands the servo motor to rotate to a specific angle and direct the product into the appropriate sorting bin.

After the product is sorted, the servo returns to its initial position and the system waits for the next product. This process is continuously repeated, providing fast, automated and efficient color-based product sorting for manufacturing applications.

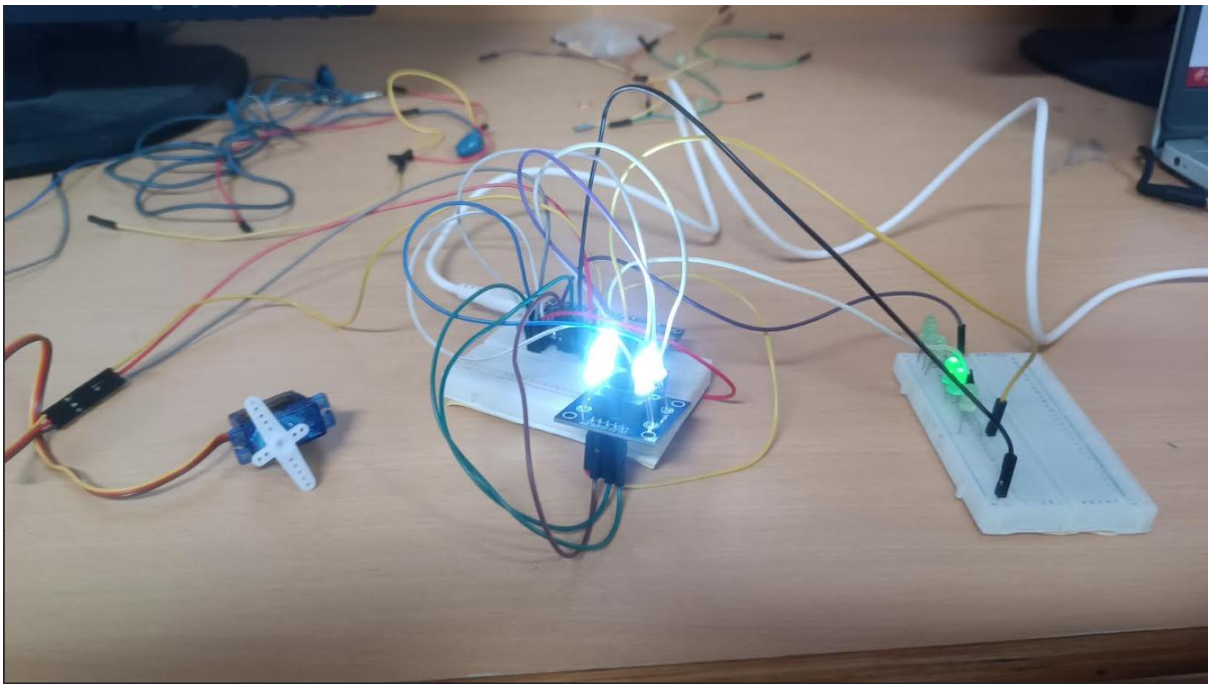
Bill Of Material:

S. no	Component	Specification	Quantity	Cost
1	ESP8266	Node MCU ESP8266	1	Rs. 450
2	Color Sensor	TCS3200	1	Rs.150
3	Servo Motor	L298N	1	Rs. 150
4	Bread Board	Standard 830-point	1	Rs. 100
5	Jumper Wire	Male-Male / Male-Female	1 set	Rs. 50
6	LED	5 mm	1 set	Rs. 50
		Total		Rs. 950

Prototype Implementation:

- The prototype was implemented using an ESP32 microcontroller, TCS3200 color sensor, IR sensor, SG90 servo motor, OLED display, conveyor belt and indicator LEDs. The IR sensor detects the presence of a product on the conveyor and triggers the color detection process. The TCS3200 sensor then measures the RGB values of the product and sends the data to the ESP32 for processing.

- The ESP32 compares the detected RGB values with predefined color thresholds and identifies the product color. The detected color and sorting status are displayed on the OLED display, while the corresponding LED provides visual indication. Based on the identified color, the ESP32 controls the SG90 servo motor, which rotates to the required angle and directs the product into its respective sorting bin.
- After the product is sorted, the servo returns to its initial position and the conveyor continues moving the next product toward the detection area. The prototype successfully demonstrates automatic color detection, servo-based sorting and continuous product classification, reducing manual effort and improving sorting efficiency.



Results and Performance:

The developed Automated Color-Based Product Sorting System successfully detected and classified products based on their color using the TCS3200 color sensor and ESP32. The IR sensor accurately detected the presence of products on the conveyor, after which the color sensor measured the RGB values and identified the corresponding color. Based on the detected color, the ESP32 controlled the servo

motor to direct the product into the appropriate sorting bin. The OLED display provided real-time information about the detected color and sorting status, while the LEDs indicated the corresponding color.

Performance Analysis

Parameter	Result
Product Detection	Successfully detected using IR sensor
Color Detection	Successfully identified using TCS3200
Color Processing	Real-time ESP32 processing
Sorting Mechanism	Servo motor controlled automatically
Product Classification	Successfully sorted into respective bins
Display	Real-time OLED indication
Visual Indication	Color-specific LED
Automation	Continuous sorting operation
Response	Fast and automatic
Manual Intervention	Significantly reduced

Overall, the prototype demonstrated a low-cost, automated and efficient color-based sorting system. It can improve sorting speed, accuracy and productivity while reducing manual labor in manufacturing industries.

